



CARE-ABLE

Maintains Details Document

VISION

A platform enabling formal recognition of hidden carers' informal caregiving capabilities

Contents

1. Purpose of the Document.....	4
2. System Overview	4
3. Hardware and Software Requirements	6
3.1 Minimum Hardware Requirements.....	6
3.2 Software Requirements.....	6
3.3 Supported Operating Systems.....	7
3.4 Browser Compatibility.....	7
4. Installation Guide.....	8
4.1 Prerequisites.....	8
4.2 Installing Node.js	8
4.3 Installing MySQL	9
4.4 Installing Required Packages.....	10
4.5 Database Setup.....	11
4.6 Running Backend Server	12
4.7 Running Frontend Application.....	13
4.8 Building for Production	14
5. Dependency Management.....	14
5.1 List of Dependencies.....	14
5.2 Version Management	15
5.3 Updating Dependencies	16
6. Database Maintenance	16
6.1 Database Structure	16
6.2 Backup Procedures.....	17
6.3 Restore Procedures.....	17
6.4 Database Optimization.....	18
6.5 Migration Handling.....	19
7. System Maintenance Procedures	19
7.1 System Monitoring.....	19

7.2 Software Updates	20
----------------------------	----

1. Purpose of the Document

The purpose of this document is to provide comprehensive maintenance, deployment, and operational guidance for the CareAble system. This document is intended to support developers, system administrators, and future maintainers in understanding, deploying, managing, updating, and extending the application efficiently. It includes detailed information regarding system architecture, installation procedures, hardware and software requirements, deployment strategies, dependency management, security considerations, troubleshooting procedures, and maintenance activities.

The document also serves as a technical reference for ensuring the long-term sustainability and reliability of the CareAble platform. It provides step-by-step instructions for configuring the development and production environments, maintaining the database, updating software components, and resolving common system issues. In addition, the document explains how future developers can extend the system functionality by integrating new modules, APIs, and user features.

The CareAble system is designed as a low-cost, scalable, and maintainable web-based platform intended primarily for Australian users with an expected user base of fewer than 10,000 users. The solution emphasizes data redundancy, security, and ease of deployment while utilizing modern web technologies and cloud infrastructure.

2. System Overview

CareAble is a web-based caregiver management and assessment platform developed to streamline caregiver registration, onboarding, assessment management, employer interactions, and administrative operations. The system provides a centralized environment where caregivers can create profiles, complete assessments, manage certifications, and interact with employers through a secure and user-friendly interface.

The system follows a client-server architecture using a React.js frontend, Node.js and Express.js backend, and a MySQL relational database. Communication between the

frontend and backend is handled through RESTful APIs using JSON data exchange. The application is designed using a three-tier architecture consisting of the presentation layer, business logic layer, and data layer.

The frontend interface is responsible for presenting responsive and interactive user interfaces for caregivers, employers, and administrators. The backend server processes application logic, authentication, authorization, assessment handling, and API requests. The MySQL database securely stores user information, assessments, onboarding details, reports, and system records.

The solution is designed for deployment on cloud infrastructure providers such as Amazon Web Services (AWS) using low-cost or free-tier services where possible. Infrastructure as Code (IaC) principles can be used to automate deployment and configuration management. The production environment prioritizes affordability, maintainability, and data redundancy to ensure reliable operation without requiring enterprise-level infrastructure costs.

The CareAble system supports multiple user roles, including:

- Caregivers
- Employers
- Administrators

Key features of the system include:

- Secure user authentication and authorization
- Caregiver onboarding and profile management
- Assessment and quiz management
- Certificate generation
- Employer management
- Administrative dashboards
- Report generation
- REST API integration
- Cloud deployment support

The system is intended for Australia-based users and is optimized for modern web browsers and cross-platform accessibility.

3. Hardware and Software Requirements

The CareAble system requires both hardware and software components to support development, testing, deployment, and production operations.

3.1 Minimum Hardware Requirements

The minimum hardware requirements are suitable for development environments and small-scale deployments with limited concurrent users.

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	20 GB available disk space
Network	Stable internet connection
Display Resolution	1366 × 768

3.2 Software Requirements

The following software technologies and tools are required for developing, deploying, and maintaining the CareAble system.

Software	Purpose
Node.js	Backend runtime environment
Express.js	Backend web framework
React.js	Frontend user interface
MySQL Server	Relational database management
MySQL Workbench	Database administration
Visual Studio Code	Source code editor

Git & GitHub	Version control and repository management
Postman	API testing
npm	Dependency/package management
Docker (Optional)	Containerized deployment
AWS Cloud Services	Cloud hosting and deployment

3.3 Supported Operating Systems

The CareAble system supports the following operating systems for both development and deployment environments.

Operating System	Support Status
Windows 10/11	Fully Supported
Ubuntu Linux 20.04+	Fully Supported
macOS Monterey or later	Fully Supported

3.4 Browser Compatibility

The CareAble web application is designed to support modern web browsers with responsive user interface compatibility.

Browser	Support Status
Google Chrome	Fully Supported
Mozilla Firefox	Fully Supported

Microsoft Edge	Fully Supported
Safari	Supported
Opera	Supported

4. Installation Guide

4.1 Prerequisites

Before installing the CareAble system, ensure that the following prerequisites are available on the machine:

- Internet connection for downloading packages and dependencies
- Administrative privileges on the operating system
- Basic knowledge of command-line operations
- Installed web browser such as Google Chrome or Mozilla Firefox
- Git installed for source code management (optional but recommended)

The following software must be installed prior to running the system:

- Node.js
- npm (Node Package Manager)
- MySQL Server
- Visual Studio Code or another code editor

4.2 Installing Node.js

Node.js is required to run the backend server and manage frontend dependencies.

Step 1: Download Node.js

Visit the official Node.js website and download the latest Long-Term Support (LTS) version.

Recommended version:

- Node.js v18 or higher

Step 2: Install Node.js

Run the installer and follow the installation wizard instructions.

Ensure the following options are enabled during installation:

- Node.js runtime
- npm package manager
- Add to system PATH

Step 3: Verify Installation

Open the terminal or command prompt and execute the following commands:

```
node
```

-v

```
npm -v
```

If successfully installed, the system will display the installed versions.

Example:

```
v18.17.0
```

```
9.6.7
```

4.3 Installing MySQL

MySQL is used as the relational database management system for storing application data.

Step 1: Download MySQL

Download MySQL Community Server from the official MySQL website.

Recommended version:

- MySQL Server 8.0+

Step 2: Install MySQL Server

Run the installer and select:

- MySQL Server
- MySQL Workbench

During installation:

- Configure the root password
- Use the default port: 3306
- Enable MySQL as a Windows/Linux service

4.4 Installing Required Packages

After cloning or downloading the project source code, install all required dependencies for both frontend and backend applications.

Backend Package Installation

Navigate to the backend project directory:

```
cd backend
```

Install dependencies:

```
npm install
```

This command installs required backend packages such as:

- Express.js
- MySQL2
- dotenv
- bcrypt
- cors
- multer

Frontend Package Installation

Navigate to the frontend project directory:

```
cd frontend
```

Install frontend dependencies:

```
npm install
```

This installs packages such as:

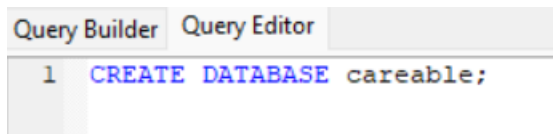
- React.js
- React Router DOM
- Axios
- Tailwind CSS
- React Icons

4.5 Database Setup

The database must be created before running the application.

Step 1: Create Database

Login to MySQL and execute:



Step 2: Select Database

“USE careable”;

Step 3: Import Database Tables

Import the provided SQL database script:

```
mysql -u root -p careable_db < careable.sql
```

Alternatively, use MySQL Workbench:

1. Open MySQL Workbench
2. Select the database

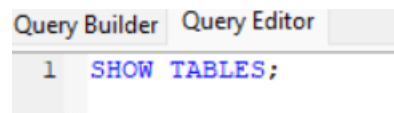
3. Open the SQL file
4. Execute the script

The script will create all required tables such as:

- caregivers
- employers
- users
- assessments
- caregiver_questions
- certificates
- onboarding_details

Step 4: Verify Database Tables

Run:



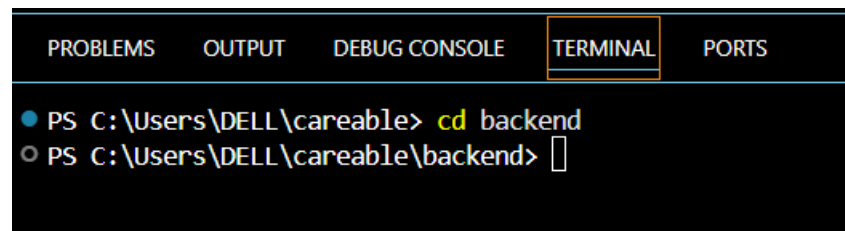
```
Query Builder  Query Editor
1  SHOW TABLES;
```

Ensure all tables are successfully created.

4.6 Running Backend Server

The backend server handles API requests, authentication, database operations, and business logic.

Navigate to the backend folder:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\DELL\careable> cd backend
PS C:\Users\DELL\careable\backend> 
```

Start the backend server:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\DELL\careable> cd backend
○ PS C:\Users\DELL\careable\backend> npm start
```

For development mode:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\DELL\careable> cd backend
○ PS C:\Users\DELL\careable\backend> npm run dev
```

If the server starts successfully, the terminal will display:

```
> backend@1.0.0 start
> node server.js

Server running on port 5000
Connected to the MySQL database.
MySQL Connected...

```

The backend API will be accessible at:

<http://localhost:5000>

4.7 Running Frontend Application

The frontend application provides the graphical user interface for users.

Navigate to the frontend folder:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\DELL\careable> cd front
○ PS C:\Users\DELL\careable\front>
```

Start the React development server:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\DELL\careable> cd front
○ PS C:\Users\DELL\careable\front> npm start
```

The application will automatically open in the default web browser.

4.8 Building for Production

To deploy the application in a production environment, optimized production builds must be generated.

Building the Frontend

Navigate to the frontend directory:

```
“cd front”
```

Generate the production build:

```
“npm run build”
```

This creates an optimized build folder containing static production files.

Running Backend in Production

Install process manager PM2 globally:

```
“npm install -g pm2”
```

Start the backend server using PM2:

```
“pm2 start server.js”
```

Useful PM2 commands:

```
pm2 list
```

```
pm2 restart all
```

```
pm2 stop all
```

5. Dependency Management

5.1 List of Dependencies

The following dependencies are used within the CareAble system to support application functionality.

Dependency	Purpose
React.js	Frontend user interface development
React Router DOM	Client-side routing

Axios	API communication
Tailwind CSS	User interface styling
Node.js	Backend runtime environment
Express.js	Backend web framework
MySQL2	Database connectivity
dotenv	Environment variable management
bcrypt	Password hashing
jsonwebtoken (JWT)	Authentication and authorization
cors	Cross-Origin Resource Sharing
multer	File upload handling
nodemon	Development server auto-reload
PM2	Production process management

5.2 Version Management

Version management ensures compatibility and prevents conflicts between software packages. The CareAble system uses the package.json and package-lock.json files to maintain dependency versions.

package.json

The package.json file contains:

- Project metadata
- Dependency lists
- Scripts
- Version numbers

```
{  
  "name": "backend",  
  "version": "1.0.0",  
  "main": "index.js",
```

5.3 Updating Dependencies

Dependencies should be updated regularly to:

- Fix security vulnerabilities
- Improve performance
- Maintain compatibility
- Receive bug fixes

Checking Outdated Packages

Run the following command:

```
PS C:\Users\DELL\careable> cd backend  
PS C:\Users\DELL\careable\backend> npm outdated
```

Updating Packages

Update packages to latest compatible versions:

```
PS C:\Users\DELL\careable> cd backend  
PS C:\Users\DELL\careable\backend> npm update
```

6. Database Maintenance

6.1 Database Structure

The CareAble system uses a relational database structure implemented using MySQL. The database consists of multiple interconnected tables designed to maintain data consistency and support application functionality.

Key database tables include:

Table Name	Purpose
users	Stores authentication and login details
caregivers	Stores caregiver profile information
employers	Stores employer details
assessments	Stores caregiver assessment data
caregiver_questions	Stores quiz and assessment answers
certificates	Stores generated certificate records
onboarding_details	Stores caregiver onboarding information

The database structure uses:

- Primary keys
- Foreign key relationships
- Data normalization
- Indexed columns for optimized queries

6.2 Backup Procedures

Regular database backups are necessary to prevent data loss caused by hardware failure, software corruption, accidental deletion, or cyber-attacks.

Manual Backup Procedure

Run the following command:

```
mysqldump -u root -p careable_db > careable_backup.sql
```

This command exports the entire database into a SQL backup file.

6.3 Restore Procedures

Database restoration is required when recovering from system failures or restoring lost data.

Restore Database from Backup

Execute the following command:

```
mysql -u root -p careable_db < careable_backup.sql
```

Verification Steps

After restoration:

- Verify database connectivity
- Check table integrity
- Validate application functionality
- Test user login and core operations

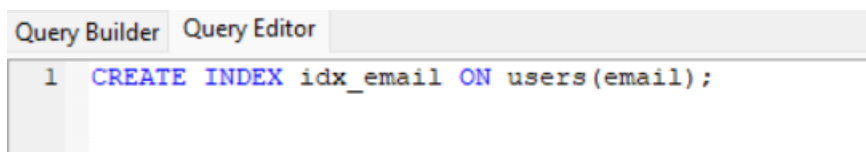
6.4 Database Optimization

Database optimization improves query performance and system responsiveness.

Optimization strategies used in the CareAble system include:

- Indexing frequently searched columns
- Reducing duplicate data
- Optimizing SQL queries
- Using normalized table structures
- Limiting unnecessary joins
- Pagination for large datasets

Example optimization techniques:

A screenshot of a database management interface. At the top, there are two tabs: 'Query Builder' and 'Query Editor'. The 'Query Editor' tab is active. Below the tabs, there is a text area containing a SQL command: '1 CREATE INDEX idx_email ON users(email);'. The text is color-coded: '1' is grey, 'CREATE' is blue, 'INDEX' is blue, 'idx_email' is blue, 'ON' is blue, 'users' is blue, and '(email);' is blue.

```
Query Builder Query Editor
1 CREATE INDEX idx_email ON users(email);
```

Additional optimization practices:

- Monitoring slow queries
- Cleaning unused records
- Archiving historical data
- Regular database health checks

6.5 Migration Handling

Database migrations are used to manage structural changes such as:

- Adding new tables
- Modifying columns
- Creating indexes
- Updating relationships

Migration handling ensures that database changes are applied consistently across development, testing, and production environments.

Recommended migration practices:

- Maintain version-controlled SQL scripts
- Test migrations before deployment
- Backup database before migration
- Document schema changes

7. System Maintenance Procedures

7.1 System Monitoring

Continuous system monitoring is necessary to ensure that the application servers, database services, and APIs are functioning correctly.

Monitoring activities include:

- Checking server uptime
- Monitoring CPU and memory usage
- Monitoring database performance
- Checking API response times
- Reviewing application logs
- Detecting unusual activities or errors

Recommended monitoring tools:

- PM2 monitoring
- AWS CloudWatch

- MySQL Workbench monitoring
- Server log analysis tools

7.2 Software Updates

Software updates are necessary to maintain security, compatibility, and application performance.

Update procedures include:

- Updating npm dependencies
- Updating Node.js versions
- Updating database software
- Applying security patches
- Updating frontend libraries

Before applying updates:

1. Create a full system backup
2. Test updates in a development environment
3. Verify compatibility with existing features
4. Review package release notes